

LEARNING ROADMAP

High Level Design (System Design) Roadmap

Designing Scalable Systems That Handle Millions

Intensive

25-30 hrs/week

Balanced

12-15 hrs/week

Flexible

6-10 hrs/week

■ 7 Phases ★ Intermediate to Advanced

<https://crio.do>

Overview

System Design is about building systems that work at scale – millions of users, petabytes of data, 99.99% uptime. Unlike LLD (which asks 'how do classes interact?'), HLD asks 'how do services, databases, and infrastructure work together?' It's tested in senior engineering interviews at every major tech company. This roadmap takes you from understanding basic components to confidently designing systems like Twitter, Netflix, or Uber. More importantly, it teaches you to think about trade-offs – because at scale, there are no perfect solutions, only trade-offs.

Who is this for?

Software engineers preparing for senior/staff level interviews, developers who want to understand large-scale systems, architects designing production systems, or anyone who's wondered 'how does Netflix actually work?'

Prerequisites

- 2+ years of software development experience
- Understanding of basic web architecture (client-server, HTTP, APIs)
- Familiarity with at least one database (SQL or NoSQL)
- Basic understanding of networking concepts
- Experience building and deploying applications

What you'll learn

- Design systems that handle millions of users
- Make informed trade-offs between consistency, availability, and latency
- Choose appropriate databases, caches, and message queues
- Estimate system capacity and resource requirements
- Identify and resolve bottlenecks and single points of failure
- Communicate designs clearly in interviews and with teams
- Pass system design interviews at FAANG and top companies

Learning Plans

Choose a plan that fits your schedule. All plans cover the same content.

Intensive

Fast Track

25-30

hrs/week

150-

180

total hrs

Focused preparation for upcoming senior-level interviews. Practice multiple systems weekly with mock interviews.

Ideal for:

- Interview prep (6-8 weeks out)
- Senior/Staff level candidates
- Developers between jobs
- Career changers targeting FAANG

Balanced

Recommended

12-15

hrs/week

180-

225

total hrs

Steady learning while working. Apply concepts to your current systems. The recommended approach for deep understanding.

Ideal for:

- Working engineers
- Long-term interview prep
- Architects building real systems
- Tech leads expanding scope

Flexible

Self-paced

6-10

hrs/week

200-

300

total hrs

Gradual mastery with time to internalize complex concepts. Best for those without immediate interview pressure.

Ideal for:

- Busy professionals
- Those new to distributed systems
- Weekend learners
- Building foundational knowledge

Phase Schedule by Plan

Phase	Topic	Intensive	Balanced	Flexible
1	Fundamentals of Scale	Week 1	Weeks 1-2	Weeks 1-4
2	Core Building Blocks	Weeks 2-3	Weeks 3-6	Weeks 5-12
3	Distributed Systems Concepts	Week 4	Weeks 7-9	Weeks 13-18
4	System Design Interview Framework	Week 5	Weeks 10-11	Weeks 19-22
5	Classic System Design Problems	Weeks 5-6	Weeks 12-16	Weeks 23-32
6	Advanced Topics	Week 7	Weeks 17-20	Weeks 33-40
7	Interview Mastery	Week 8	Weeks 21-24	Weeks 41-48

1 Fundamentals of Scale

Intensive: Week 1

Balanced: Weeks 1-2

Flexible: Weeks 1-4

Before diving into specific systems, you need to understand the building blocks. What does 'scale' actually mean? How do you measure it? What are the fundamental challenges when systems grow? This phase establishes the vocabulary and mental models that everything else builds on.

Learning Objectives

- Understand what scalability means and how to measure it
- Differentiate between vertical and horizontal scaling
- Perform back-of-envelope calculations
- Understand latency, throughput, and availability
- Know common system design terminology

Topics to Cover

What is Scalability?

WHY: Scalability is the core challenge of system design. A system that works for 100 users may fail at 100,000. Understanding scaling strategies is fundamental to every design decision.

- Vertical scaling (scale up)
- Horizontal scaling (scale out)
- When to use each
- Cost considerations
- Elasticity and auto-scaling
- Stateless vs stateful services

Back-of-Envelope Calculations

WHY: Before designing, you need to know the scale. How much storage? How many requests per second? These rough calculations guide your architecture. Interviewers expect them.

- Traffic estimation (QPS, DAU, MAU)
- Storage estimation
- Bandwidth estimation
- Latency numbers every engineer should know
- Powers of 2 and useful approximations
- Practice exercises

Key Metrics

WHY: You can't improve what you can't measure. Understanding latency percentiles, throughput, and availability SLAs is essential for evaluating designs.

- Latency (p50, p95, p99)
- Throughput (RPS, QPS)
- Availability (nines)
- Durability
- SLAs and SLOs
- Monitoring and alerting basics

Single Points of Failure

WHY: A chain is only as strong as its weakest link. Identifying and eliminating single points of failure (SPOFs) is fundamental to reliable systems.

- Identifying SPOFs
- Redundancy patterns
- Active-passive vs active-active
- Failover mechanisms
- Graceful degradation
- Blast radius containment

Hands-on Projects

Capacity Planning Exercise

Estimate the requirements for a photo-sharing app with 10M daily users, each uploading 2 photos/day. Calculate: daily storage needs, annual storage, read/write QPS, and bandwidth requirements. Show your work.

[Back-of-envelope](#)[Storage estimation](#)[Traffic estimation](#)

Analyze a Real System

Pick a service you use (Spotify, Dropbox, Twitter). Estimate its scale: users, requests, data. Research how they actually architected it. Compare your estimates to reality.

[Estimation](#)[Research](#)[Architecture analysis](#)

Resources

[REFERENCE](#)[Latency Numbers Every Programmer Should Know](#)[COURSE](#)[ByteByteGo - Back-of-Envelope Estimation](#)[EXERCISES](#)[Napkin Math \(Simon Wilson\)](#)

Milestone



Given a product description, estimate its scale (users, storage, bandwidth, QPS) within an order of magnitude. Identify potential bottlenecks and SPOFs. Speak fluently about latency percentiles and availability nines.

2 Core Building Blocks

Intensive: Weeks 2-3

Balanced: Weeks 3-6

Flexible: Weeks 5-12

Every large system is built from the same fundamental components: load balancers, caches, databases, message queues, CDNs. This phase dives deep into each component – what it does, when to use it, and the trade-offs involved. Master these, and you can design almost any system.

Learning Objectives

- Understand load balancing strategies and when to use each
- Design effective caching strategies
- Choose between SQL and NoSQL databases
- Use message queues for asynchronous processing
- Leverage CDNs for content delivery

Topics to Cover

Load Balancing

WHY: Load balancers distribute traffic across servers, preventing any single server from being overwhelmed. They're the foundation of horizontal scaling. No scalable system exists without them.

- L4 vs L7 load balancing
- Algorithms (round-robin, least connections, IP hash)
- Health checks
- Session persistence (sticky sessions)
- Global load balancing (DNS, Anycast)
- Software vs hardware load balancers

Caching

WHY: Caching reduces latency and database load dramatically. Redis can handle 100K+ reads/sec vs MySQL's ~1K. A good caching strategy can reduce database load by 80-90%.

- Cache-aside, read-through, write-through, write-behind
- Cache invalidation strategies
- TTL and eviction policies (LRU, LFU)
- Distributed caching (Redis, Memcached)
- CDN as cache
- Cache stampede and thundering herd

Databases: SQL vs NoSQL

WHY: Database choice is one of the most important architectural decisions. SQL offers ACID and relationships; NoSQL offers flexibility and scale. Understanding trade-offs is essential.

- ACID vs BASE
- When to use SQL (transactions, relationships)
- When to use NoSQL (scale, flexibility)
- Document stores (MongoDB)
- Key-value stores (Redis, DynamoDB)
- Wide-column stores (Cassandra)
- Graph databases (Neo4j)

Database Scaling

WHY: Databases often become the bottleneck. Understanding replication, sharding, and partitioning strategies is crucial for systems that need to scale data access.

- Read replicas
- Master-slave replication
- Sharding strategies (hash, range, directory)
- Consistent hashing
- Database federation
- Vitess, CockroachDB, and NewSQL

Message Queues

WHY: Message queues decouple services and handle traffic spikes. They enable asynchronous processing, which is essential for resilient, scalable systems.

- When to use message queues
- Kafka vs RabbitMQ
- Pub/Sub patterns
- Message ordering and delivery guarantees
- Dead letter queues
- Event-driven architecture

Content Delivery Networks (CDNs)

WHY: CDNs cache static content at edge locations worldwide. They dramatically reduce latency for users far from your servers and offload traffic from your origin.

- How CDNs work
- Push vs pull CDN
- Cache invalidation
- CDN for dynamic content
- Multi-CDN strategies
- Popular CDNs (CloudFront, Akamai, Cloudflare)

Hands-on Projects

Design a Caching Layer

For a read-heavy application (90% reads, 10% writes), design a caching strategy. Decide: what to cache, invalidation strategy, TTL, cache size. Calculate hit rate needed to reduce DB load by 80%.

Caching strategies

Trade-off analysis

Capacity planning

Database Selection Exercise

Given three scenarios: (1) banking transactions, (2) social media posts, (3) real-time analytics. Choose SQL or NoSQL for each, justify your choice, and design the schema/data model.

Database selection

Data modeling

Trade-off analysis

Resources

RESOURCE [System Design Primer \(GitHub\)](#)

ARTICLE [AWS vs NoSQL \(AWS\)](#)

ARTICLE [Kafka vs RabbitMQ \(AWS\)](#)

Milestone



Given a system requirement, choose appropriate building blocks and justify your choices. Explain the trade-offs of SQL vs NoSQL, caching strategies, and when to use message queues.

3 Distributed Systems Concepts

Intensive: Week 4

Balanced: Weeks 7-9

Flexible: Weeks 13-18

At scale, systems are distributed across multiple machines, data centers, and even continents. This introduces fundamental challenges: how do you maintain consistency? What happens when the network fails? This phase covers the theoretical foundations that guide distributed system design.

Learning Objectives

- Understand the CAP theorem and its implications
- Choose appropriate consistency models
- Design for partition tolerance
- Implement consensus in distributed systems
- Handle distributed transactions

Topics to Cover

CAP Theorem

WHY: CAP theorem states you can only have 2 of 3: Consistency, Availability, Partition Tolerance. Since network partitions happen, you must choose between C and A. This fundamental trade-off shapes every distributed system.

- Consistency (all nodes see same data)
- Availability (every request gets response)
- Partition Tolerance (system works despite network failures)
- CP vs AP systems
- Real-world examples
- CAP misconceptions

PACELC Theorem

WHY: CAP only addresses partition scenarios. PACELC adds: Even without partitions, you trade off Latency vs Consistency. This more complete model helps choose between strongly consistent and eventually consistent systems.

- PACELC explained
- PA/EL systems (Dynamo, Cassandra)
- PC/EC systems (RDBMS, CockroachDB)
- Choosing based on requirements
- Latency-consistency trade-off
- Tunable consistency

Consistency Models

WHY: 'Consistency' isn't binary. Strong, eventual, causal, read-your-writes — each model has different guarantees and performance characteristics. Choosing the right one depends on your use case.

- Strong consistency
- Eventual consistency
- Causal consistency
- Read-your-writes consistency
- Session consistency
- Linearizability vs serializability

Consensus Algorithms

WHY: How do distributed nodes agree on a value when some might fail? Consensus algorithms (Paxos, Raft) solve this. They're the foundation of distributed databases and coordination services.

- The consensus problem
- Paxos (conceptual)
- Raft (more accessible)
- Leader election
- ZooKeeper and etcd
- When consensus is needed

Distributed Transactions

WHY: Sometimes you need transactions across multiple services or databases. 2PC, Saga, and other patterns handle this, each with different trade-offs.

- Two-phase commit (2PC)
- Saga pattern
- Compensating transactions
- Idempotency
- Outbox pattern
- When to avoid distributed transactions

Hands-on Projects

Consistency Analysis

Analyze three real systems: (1) a banking system, (2) a social media feed, (3) a collaborative document editor. For each, determine what consistency model is needed and why. What breaks if you choose wrong?

Consistency models

Trade-off analysis

Requirements mapping

Partition Tolerance Design

Design how a shopping cart service should behave during a network partition between data centers. Should it favor availability (let users add items) or consistency (reject operations)? Justify your choice.

CAP theorem

Failure modes

Business requirements

Resources

ARTICLE [CAP Theorem \(AlgoMaster\)](#)

BOOK [Designing Data-Intensive Applications](#)

VISUALIZATION [The Raft Consensus Algorithm](#)

Milestone



Explain CAP theorem with real examples. Given a system requirement, choose appropriate consistency model and justify it. Understand when eventual consistency is acceptable and when it's not.

4

System Design Interview Framework

Intensive: Week 5

Balanced: Weeks 10-11

Flexible: Weeks 19-22

System design interviews follow a pattern. You get a vague prompt ('Design Twitter'), and you have 45-60 minutes to produce a complete design. Most candidates fail by jumping into solutions without understanding requirements. This phase teaches a systematic framework that works for any problem.

Learning Objectives

- Follow a consistent framework for any system design problem
- Ask the right clarifying questions
- Define appropriate APIs and data models
- Draw clear high-level architecture diagrams
- Dive deep into specific components when asked
- Discuss trade-offs and alternatives

Topics to Cover

The 7-Step Framework

WHY: A framework prevents chaos. Instead of thinking about everything at once, you handle each phase methodically. This is how senior engineers actually approach design problems.

- Step 1: Clarify Requirements (5 min)
- Step 2: Estimate Scale (5 min)
- Step 3: Define API (5 min)
- Step 4: Design Data Model (5 min)
- Step 5: High-Level Design (10 min)
- Step 6: Deep Dive Components (15 min)
- Step 7: Address Bottlenecks (5 min)

Clarifying Requirements

WHY: The prompt is intentionally vague. 'Design Twitter' doesn't tell you: Read-heavy or write-heavy? Global or single region? Real-time or eventual? These questions shape your entire design.

- Functional requirements
- Non-functional requirements (scale, latency, availability)
- Constraints and assumptions
- What's in scope vs out of scope
- User journey mapping
- Sample clarifying questions

API Design

WHY: APIs define what the system does. Designing them early forces you to think about use cases concretely. It also shows the interviewer you understand the system's external interface.

- RESTful API design
- Request/response formats
- Authentication and rate limiting
- Pagination strategies
- API versioning
- GraphQL considerations

Drawing Architecture Diagrams

WHY: A picture is worth a thousand words. Clear diagrams communicate your design quickly and show you can think visually. Practice drawing while explaining.

- Essential diagram components
- Data flow arrows
- Labeling clearly
- Using color effectively
- Common diagram patterns
- Tools vs whiteboard

Deep Dive Strategies

WHY: Interviewers often ask you to 'go deeper' on a specific component. Knowing how to dive into database design, caching strategy, or specific algorithms demonstrates depth.

- Reading interviewer signals
- When to go broad vs deep
- Database schema deep dive
- Caching layer deep dive
- Handling follow-up questions
- Knowing your limits

Hands-on Projects

Timed Practice Sessions

Set a 45-minute timer. Pick a problem (URL shortener, rate limiter, etc.). Follow the framework strictly. Record yourself and review. Identify where you got stuck or went off-track.

Time management

Framework execution

Self-review

Mock Interviews

Do at least 3 mock interviews with peers or services like Pramp. Get feedback on clarity, trade-off discussion, and handling of follow-up questions. Iterate based on feedback.

Communication

Pressure handling

Incorporating feedback

Resources

GUIDE [Design Gurus - 7 Steps to Ace System Design](#)

GUIDE [Interviewing.io - System Design Guide](#)

PRACTICE [Pramp \(Free mock interviews\)](#)

Milestone



Complete 5 system design problems using the framework within 45 minutes each. Get positive feedback on at least 2 mock interviews. Be able to adapt your approach based on interviewer direction.

5

Classic System Design Problems

Intensive: Weeks 5-6

Balanced: Weeks 12-16

Flexible: Weeks 23-32

Certain problems appear in interviews repeatedly: URL shortener, rate limiter, social media feed, chat application. This phase covers the most common ones. But the goal isn't memorization – it's pattern recognition. Each problem teaches principles applicable to many others.

Learning Objectives

- Design common interview systems confidently
- Recognize patterns that apply across problems
- Handle variations and extensions
- Discuss alternatives and trade-offs
- Adapt designs to different requirements

Topics to Cover

URL Shortener (TinyURL)

WHY: The 'Hello World' of system design. Teaches: key generation, encoding, databases, caching, analytics. Simple enough to complete in an interview, complex enough to show depth.

- Short URL generation (Base62, hashing)
- Read-heavy optimization
- Database choice
- Caching strategy
- Analytics and tracking
- Custom URLs and expiration

Rate Limiter

WHY: Every API needs rate limiting. Teaches: distributed counting, sliding windows, and the challenge of coordinating limits across servers. Often asked as a deep-dive component.

- Token bucket algorithm
- Sliding window counter
- Fixed window vs sliding window
- Distributed rate limiting
- Rate limiting at different layers
- Handling bursts

Social Media Feed (Twitter/Instagram)

WHY: The classic 'design Twitter' question. Teaches: fan-out strategies, timeline generation, caching, and handling celebrities with millions of followers.

- Fan-out on write vs read
- Timeline generation
- Caching strategies for feeds
- Handling celebrities (hot users)
- Post storage and retrieval
- Real-time updates (push vs pull)

Chat Application (WhatsApp/Slack)

WHY: Real-time messaging at scale. Teaches: WebSockets, presence, message delivery guarantees, and handling offline users.

- Connection management (WebSockets)
- Message delivery guarantees
- Group messaging
- Presence and typing indicators
- Message storage
- End-to-end encryption considerations

Video Streaming (Netflix/YouTube)

WHY: Media at massive scale. Teaches: CDN usage, encoding, adaptive bitrate streaming, and content recommendation basics.

- Video processing pipeline
- Encoding and transcoding
- Adaptive bitrate streaming (HLS, DASH)
- CDN strategy
- Content recommendation
- Copyright protection (DRM)

Ride Sharing (Uber/Lyft)

WHY: Location-based services with real-time matching. Teaches: geospatial indexing, matching algorithms, and surge pricing.

- Geospatial indexing (QuadTrees, Geohash)
- Driver-rider matching
- ETA calculation
- Surge pricing
- Trip tracking
- Payment integration

Hands-on Projects

Solve All Classic Problems

Design each of the above systems from scratch. For each: clarify requirements, estimate scale, design API, draw architecture, deep-dive one component. Don't look at solutions first.

All system design skills

Pattern recognition

Diverse domains

Variation Practice

Take a solved problem and change requirements: What if URL shortener needs 99.999% availability? What if Twitter needs to support 10M concurrent users in real-time? How does your design change?

Adaptability

Trade-off analysis

Scale thinking

Resources

COURSE [ByteByteGo System Design Course](#)

COURSE [Grokking System Design Interview](#)

PRACTICE [System Design School](#)

Milestone



Design any of these systems in 45 minutes without hints. Recognize which patterns from one system apply to others. Handle variations and extensions confidently.

6

Advanced Topics

Intensive: Week 7

Balanced: Weeks 17-20

Flexible: Weeks 33-40

Senior and staff-level interviews expect depth in specific areas. This phase covers advanced topics that differentiate strong candidates: microservices, security, observability, and emerging architectures. These are also essential for building real production systems.

Learning Objectives

- Design microservices architectures
- Implement security at scale
- Build observable systems
- Handle global distribution
- Stay current with emerging technologies

Topics to Cover

Microservices Architecture

WHY: Most large systems use microservices. Understanding service boundaries, communication patterns, and the challenges of distributed systems is essential for senior roles.

- Monolith vs microservices
- Service boundaries (DDD)
- Synchronous vs asynchronous communication
- API Gateway pattern
- Service mesh (Istio, Linkerd)
- Microservices anti-patterns

Security at Scale

WHY: Security isn't optional. Authentication, authorization, encryption, and compliance are expected topics in senior interviews and critical in production.

- Authentication (OAuth, JWT, SAML)
- Authorization (RBAC, ABAC)
- Encryption in transit and at rest
- API security
- Secrets management
- Compliance considerations (GDPR, SOC2)

Observability

WHY: You can't run what you can't see. Logging, metrics, and tracing are how you understand and debug production systems. They're also how you detect and respond to incidents.

- Three pillars: logs, metrics, traces
- Distributed tracing (Jaeger, Zipkin)
- Metrics and alerting (Prometheus, Grafana)
- Log aggregation (ELK, Loki)
- SLIs, SLOs, and error budgets
- Incident response

Global Distribution

WHY: Truly global systems must handle users on every continent with low latency. Multi-region architecture, data replication, and conflict resolution are advanced but important topics.

- Multi-region architecture
- Active-active vs active-passive
- Data replication across regions
- Conflict resolution (CRDTs, vector clocks)
- Global load balancing
- Latency optimization

Emerging Technologies

WHY: The field evolves constantly. Serverless, edge computing, and AI-driven architectures are changing how we build systems. Awareness shows you're current.

- Serverless architecture (Lambda, Functions)
- Edge computing
- AI/ML infrastructure
- Event sourcing and CQRS
- GraphQL at scale
- Real-time data platforms

Hands-on Projects

Microservices Decomposition

Take a monolithic e-commerce system (users, products, orders, payments). Design how to decompose it into microservices. Define service boundaries, communication patterns, and data ownership.

Microservices

DDD

Service design

Observability Design

For a distributed system with 50 microservices, design the observability stack. What metrics do you collect? How do you trace requests across services? How do you alert on problems?

Observability

Monitoring

Incident response

Resources

BOOK

[Building Microservices \(Sam Newman\)](#)

BOOK

[Google SRE Book](#)

BOOK

[DDIA \(Designing Data-Intensive Applications\)](#)

Milestone



Discuss microservices trade-offs intelligently. Design an observability strategy for a distributed system. Understand when serverless is appropriate. Show depth in at least one advanced area.

7 Interview Mastery

Intensive: Week 8

Balanced: Weeks 21-24

Flexible: Weeks 41-48

Knowing the content isn't enough – you need to perform under pressure. This phase focuses on interview skills: communication, handling ambiguity, discussing trade-offs, and building confidence through practice. The goal is to move from 'knows the material' to 'would hire confidently.'

Learning Objectives

- Communicate designs clearly and confidently
- Handle ambiguity and open-ended questions
- Discuss trade-offs without prompting
- Recover gracefully from mistakes
- Demonstrate senior-level thinking

Topics to Cover

Communication Skills

WHY: Technical skills mean nothing if you can't communicate them. Interviewers evaluate your ability to explain complex concepts clearly. This is a learned skill.

- Thinking out loud
- Structuring explanations
- Using diagrams effectively
- Adjusting to interviewer level
- Handling interruptions
- Summarizing key points

Trade-off Discussion

WHY: Senior engineers don't just propose solutions – they discuss alternatives. Proactively discussing trade-offs shows maturity and depth. Don't wait to be asked.

- Identifying trade-offs
- Presenting alternatives
- Justifying decisions
- Acknowledging limitations
- When to revisit decisions
- Trade-off vocabulary

Handling Ambiguity

WHY: Real problems are ambiguous. Interviewers want to see how you handle uncertainty. Making reasonable assumptions and stating them clearly is a strength, not a weakness.

- Making assumptions explicit
- Scoping appropriately
- Asking good questions
- When to decide vs when to ask
- Pivoting based on feedback
- Comfortable with 'I don't know'

Common Pitfalls

WHY: Knowing what trips up most candidates helps you avoid those mistakes. These are patterns we've seen repeatedly in failed interviews.

- Jumping to solutions
- Over-engineering
- Ignoring non-functional requirements
- Poor time management
- Not engaging the interviewer
- Freezing under pressure

Building Confidence

WHY: Confidence comes from preparation and practice. The more problems you solve, the more mock interviews you do, the more comfortable you become. There are no shortcuts.

- Deliberate practice
- Learning from failures
- Mock interview cadence
- Mental preparation
- Physical preparation
- Interview day strategies

Hands-on Projects

Mock Interview Marathon

Do 10 mock interviews over 2 weeks. Use Pramp, interviewing.io, or peers. Get written feedback on each. Track improvement areas. Iterate until you consistently receive positive signals.

Interview performance

Feedback incorporation

Confidence building

Teach to Learn

Explain a system design (e.g., design Netflix) to someone non-technical. Then explain it to an engineer unfamiliar with the topic. Teaching reveals gaps in your understanding.

Communication

Deep understanding

Simplification

Resources

PRACTICE

Pramp (Free mock interviews)

PRACTICE

interviewing.io

PRACTICE

Exponent (Paid mock interviews)

Milestone



Pass 3+ mock interviews with positive feedback. Handle ambiguous problems without freezing. Proactively discuss trade-offs. Feel confident walking into any system design interview.

Your Path Forward

Career Opportunities

✓ Pass system design interviews at FAANG and top companies

✓ Design scalable systems in your current role

✓ Progress to senior, staff, or principal engineer levels

✓ Lead architecture discussions with confidence

✓ Mentor others on system design

Tips for Success

- Start with requirements, not solutions – the prompt is intentionally vague
- There are no perfect designs, only trade-offs – discuss them proactively
- Draw while you talk – visual communication is powerful
- Practice back-of-envelope calculations until they're automatic
- Know when to go broad vs deep – follow the interviewer's lead
- Understand WHY components exist, not just WHAT they are
- Mock interviews are the best preparation – do many of them
- Read engineering blogs from Netflix, Uber, and Airbnb for real-world context
- Don't memorize solutions – understand patterns that transfer across problems
- Confidence comes from practice, not from reading more material

Next Steps

1. Memorize latency numbers and powers of 2
2. Pick one classic problem and design it this week
3. Read one engineering blog post about a real system
4. Schedule your first mock interview
5. Draw the architecture of a system you use daily
6. Start a study group with peers preparing for interviews

Ready to Start Your Journey?

Everything in this roadmap is free to learn on your own. But if you'd like structured guidance, hands-on projects, and mentor support – we'd love to have you at Crio.Do

Explore Crio.Do

<https://crio.do>